DATE / /

Database Relationships and SQL Joins.

1. Database Relationship.

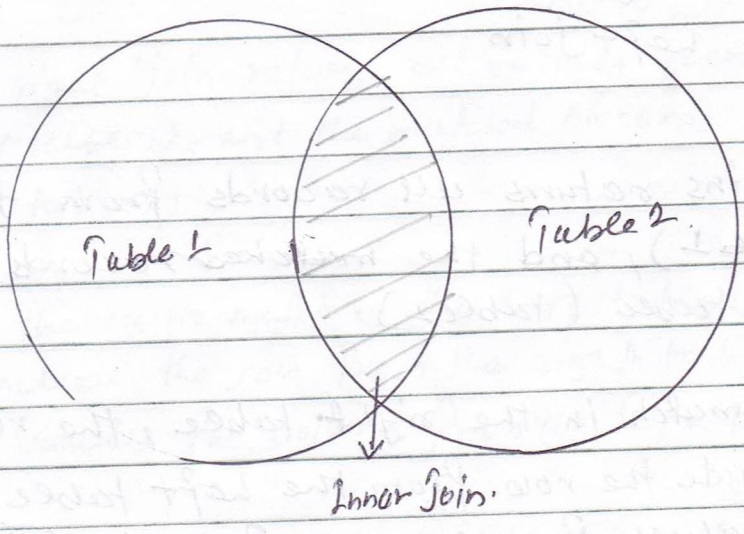
↳ One-to-Many Relationship.

→ A row in one table can be related to multiple rows in another table.

→ Example: A customer can have multiple purchases, but each purchase belongs to only one customer.

↳ Primary key & Foreign key: already done

Inner Joins



→ The inner join selects records that have matching values in both tables

→ Use when you want all records from the ~~right table~~ have matching values in both tables.

→ Example: List only customers who have made purchases and their purchase details.

→ If there's no match in either table, the row is excluded from the result.

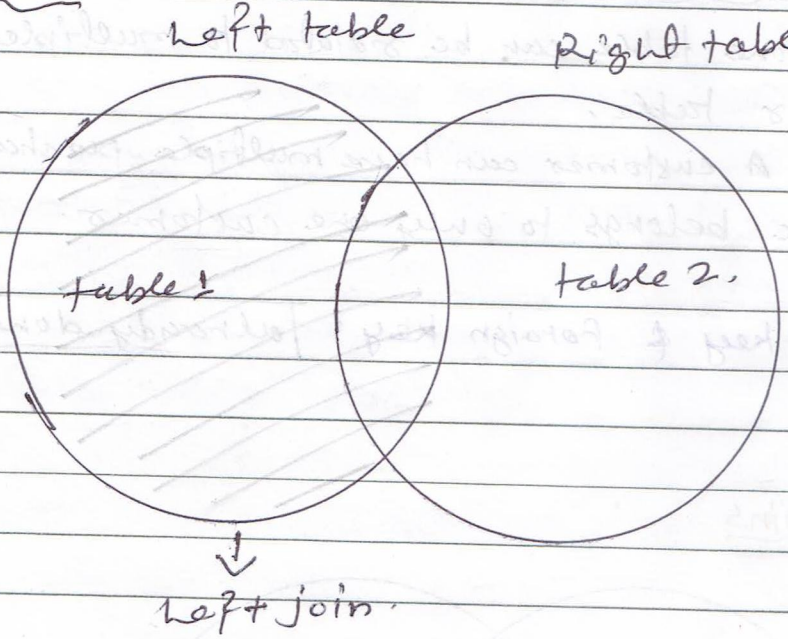
PAGE NO.
DATE / /

Syntax

```
select columns
from table1
Inner Join table2
on table1.column-name = table2.column-name;
```

table1.column-name ↓ Foreign key
table2.column-name ↓ primary key.

Left Joins



→ the left joins returns all records from the left table (table1), and the matched records from the right table (table2).

→ if there's no match in the right table, the result will still include the row from the left table, but with Null values for columns from the right table.

→ Example: list all customer and their purchases including customer who haven't made any purchases.

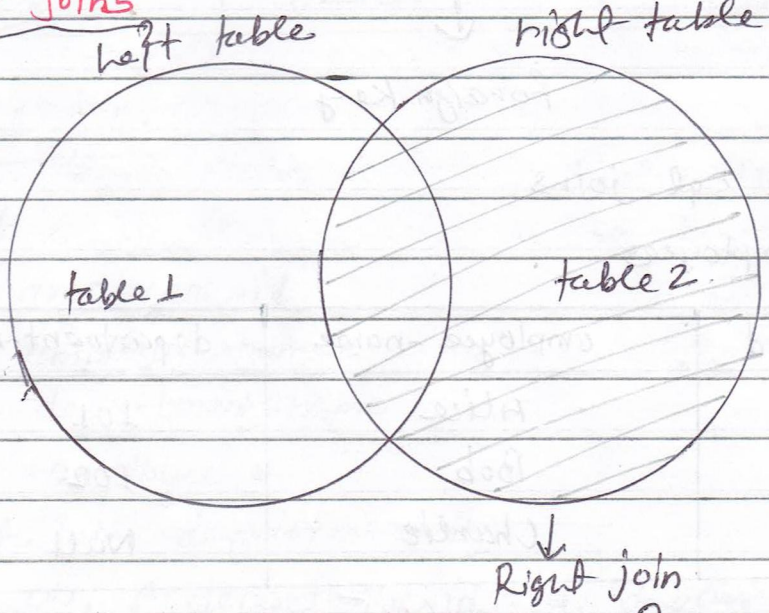
PAGE NO.....
DATE / /

Syntax

```
select columns
from table1
left join table2
on table1.column_name = table2.column_name;
```

Foreign key ↓ | Primary key ↓

Right Joins



→ The right join returns all records from the right table (table2), and the matched records from the left table (table1).

→ If there's no match in the left table, the result will still include the row from the right table, but with [Null] values for columns from the left table.

→ Example: list all purchase and their associated customers, including purchase that might not have a linked customer (though this is rare in well-designed databases).

Syntax

S.

PAGE NO.....
DATE / /

Syntax

Select columns
From table1
Right join table2
On table1.column-name = table2.column-name;

↓ Foreign key

↓ primary key.

database: sql_joins

table: employees

employee_id	employee_name	department_id
1	Alice	101
2	Bob	102
3	Charlie	Null

Int primary varchar(50) Int

table: department

department_id	department_name
101	HR
102	Finance
103	IT

Int primary varchar(100)

Left Join

Select
e.employee_id,
e.employee_name,
d.department_name
From employee e
Left Join department d
On e.department_id = d.department_id;

PAGE NO.....
DATE / /

Left joins

output

employee_id	employee_name	department_name
1	Alice	HR
2	Bob	Finance
3	Charlie	Null

Right_joins

```
Select
    e.employee_id,
    e.employee_name,
    d.department_name
from employee e
Right Join department d
    on e.department_id = d.department_id;
```

output

employee_id	employee_name	department_name
1	Alice	HR
2	Bob	Finance
Null	Null	IT

Inner Join

```
Select
    e.employee_id,
    e.employee_name,
    d.department_name
from employee e
Inner Join department d
    on e.department_id = d.department_id;
```

PAGE NO.....
DATE / /

out

employee_id	employee_name	department_name
1	Alice	HR
2	Bob	Finance

4 Common Pitfall When Filtering Joined Data.

customer_id	customer_name
1	Alice Johnson
2	Bob Smith.
3	Charlie White.
4	Diana Brown.

Int Primary key varchar(50).

database: ~~customers1~~ sql_joins

tablename: customers1

purchase_id	customer_id	market_date	purchase_amount
1	1	2025-01-15	50.00
2	1	2025-01-16	75.00
3	2	2025-01-15	100.00

Int Int Date Decimal(10,2).

auto_increment database: sql_joins.

primary key tablename: customer_purchases1

2nd this table(Foreign key (customer_id) References customers1 (customer_id))

PAGE NO.....
DATE / /

when using [Left Join] in SQL, the goal is to return all records from the "Left" table (the 1st table in the join), even if there are no matching records in the right table, the result will still include the left table's records, but with [Null] values for the columns from the right table.

A common pitfall occurs when you add a [where] clause that filters on a column from the right table. If the filter excludes [Null] values, it effectively removes all records from the left table that don't have a match in the right table. This behaviour transforms the [Left Join] into something similar to an [Inner Join] which only returns records that have matches in both tables.

To avoid this, you need to explicitly account for [Null] values in the [Where] clause when filtering on columns from the right table. This ensures that records from the left table without matches in the right table are still included in the result.

(1)
```
Select c.customer_id, c.customer_name, cp.market_date
from customers as c
Left Join customer_purchases as cp
    on c.customer_id = cp.customer_id
where cp.market_date <> '2025-01-15';
```

Problems

This query excludes customers who did not make a purchase on '2025-01-15' and behaves like an Inner Join, unintentionally removing customers without purchases.

PAGE NO.....
DATE / /

(2) Select Distinct c.customer_id, c.customer_name
from customers as c
Left Join customer_purchases as cp
on c.customer_id = cp.customer_id.
where (cp.market_date <> '2025-01-15' or
cp.market_date IS NULL);

why This works

↳ (cp.market_date <> '2025-01-15' or cp.market_date
) ensures that customers with no purchases are
also included.

↳ Distinct removes duplicate rows for customers
with multiple purchases.

(1) output

customer_id	customer_name	market_date
1	Alice Johnson	2025-01-16.

(2) output

customer_id	customer_name
1	Alice Johnson
3	Charlie White
4	Diana Brown.